

协议栈和网卡

文档类型 Formal Document

日期 2026-06-11

目录

创建套接字

协议栈内部

套接字的实体

调用socket时的操作

连接服务器

连接的含义

负责保存控制信息的头部

连接操作的实际过程

收发数据

将 HTTP 请求消息交给协议栈

对较大的数据进行拆分

使用 ACK 号确认网络包收到

根据网络包平均往返时间调整 ACK 号等待时间

使用窗口有效管理 ACK 号

ACK 与窗口的合并

接收 HTTP 响应消息

删除套接字

数据发送完毕后断开连接

删除套接字

收发小结

IP与以太网的包收发

包的基本知识

包收发操作

生成包含接收方 IP 地址的 IP 头部

生成以太网用的 MAC 头部

通过 ARP 查询目标路由器的 MAC 地址

以太网的基本知识

将 IP 包转换成电或光信号发送出去

给网络包再加 3 个控制数据

向集线器发送网络包

接收返回包

将服务器的响应包从 IP 传递给 TCP

UDP 协议收发

不需要重发的数据用 UDP 发送更高效

控制用的短数据

音频和视频数据

协议栈和网卡

使用电信号传输TCP/IP数据。

<figure> </figure>

操作系统中的网络控制软件叫作**协议栈**。这部分研究协议栈的内部。这个软件会将从浏览器接收到的消息打包，然后加上目的地址等信息（给包裹贴上运输单）。

这个软件还有其他一些功能，例如当发生通信错误时重新发送包，或者调节数据发送的速率等，全自动快递员。协议栈会将包交给网卡(负责以太网或无线网络通信的硬件)，网卡会将包转换为电信号并通过网线发送出去。

这部分就是关于协议栈内部的事情，关于协议栈如何用电信号传输TCP/IP数据。分成这几部分

- 创建套接字
- 连接服务器
- 收发数据
- 从服务器断开连接并删除套接字
- IP与以太网的包收发操作
- 用UDP协议收发数据的操作

创建套接字

协议栈内部

操作系统里的网络控制软件（协议栈）和网络硬件（网卡）。

协议栈的工作从表面上是看不到的，在探索之前，先简单看看协议栈里有些啥。

<figure> </figure>

最上面是网络应用程序，他们的数据收发工作会委托给下层来完成。应用程序的下面是Socket库，其中就有解析器，查找域名对应的IP的那个功能。

应用程序的下一层是操作系统，其中包括协议栈。协议栈的上半部分有两块，分别是负责用TCP协议收发数据的部分和负责用UDP协议收发数据的部分，它们会接受应用程序的委托执行收发数据的操作。下面一半是用IP协议控制网络包收发操作的部分。在互联网上传送数据时，数据会被切分成一个一个的网络包，而将网络包发送给通信对象的操作就是由IP来负责的。此外，IP中还包括ICMP协议和ARP协议。ICMP用于告知网络包传送过程中产生的错误以及各种控制消息，ARP用于根据IP地址查询相应的以太网MAC地址。

再下一层驱动程序，负责控制网卡硬件。最下层硬件的网卡则负责完成实际的收发操作，也就是对网线中的信号执行发送和接收的操作。

- TCP: Transmission Control Protocol
- UDP: User Datagram Protocol
- ICMP: Internet Control Message Protocol
- ARP: Address Resolution Protocol
- IP: Internet Protocol

TCP和UDP：像浏览器、邮件等一般的应用程序都是使用TCP收发数据的，而像DNS查询等收发较短的控制数据的后面细说。

网络包：网络中的数据会被切分成几十字节到几千字节的小块，每一个小数据块被称为一个包。后面也细说。

套接字的实体

协议栈是根据套接字中记录的控制信息来工作的。

在数据收发中扮演关键角色的套接字，让我们来看看它具体是个怎样的东西。

在协议栈内部有一块用于存放控制信息的内存空间，这里记录了用于控制通信操作的控制信息，例如通信对象的IP地址、端口号、通信操作的进行状态等。本来套接字就只是一个概念而已，并不存在实体，如果一定要赋予它一个实体，我们可以说这些控制信息就是套接字的实体，或者说存放控制信息的内存空间就是套接字的实体。

协议栈在执行操作时需要参阅这些控制信息。例如，在发送数据时,需要看套接字中的通信对象IP地址和端口号，以便向指定的IP地址和端口发送数据。在发送数据之后，协议栈需要等待对方返回收到数据的响应信息，但数据也可能在中途丢失，永远也等不到对方的响应。在这样的情况下，我们不能一直等下去，需要在等待一定时间之后重新发送丢失的数据，这就需要协议栈能够知道执行发送数据操作后过了多长时间。为此，套接字中必须要记录是否已经收到响应，以及发送数据后经过了多长时间，才能根据这些信息按照需要执行重发操作。

上面说的只是其中一个例子。套接字中记录了用于控制通信操作的各种控制信息，协议栈则需要根据这些信息判断下一步的行动，这就是套接字的作用。

windows中使用

```
netstat -ano
```

命令，可以看到套接字的信息。

调用socket时的操作

调用Socket库里的程序组件时，协议栈内部时怎么工作的。

再看看从应用程序出发，进行网络通信时做的一系列事情。

<figure> </figure>

首先是创建套接字的阶段，应用程序调用 `socket()` 申请创建套接字，协议栈根据传入的参数进行创建套接字的操作。在这个过程中，协议栈首先会分配用于存放一个套接字所需的内存空间，然后向其写入初始状态。

`socket()` 组件返回这个套接字的描述符。

应用程序在向协议栈进行收发数据委托时要提供这个描述符。套接字中记录了通信双方的信息以及通信处于怎样的状态，所以只要通过描述符确定了相应的套接字，协议栈就能够获取所有的相关信息，这样一来，应用程序就不需要每次都告诉协议栈应该和谁进行通信了。

连接服务器

连接的含义

创建套接字之后，应用程序（浏览器）就会调用 `connect()`，随后协议栈会将本地的套接字与服务器的套接字进行连接。

以太网的网线都是一直连接的状态，那么这里的“连接”到底是什么意思呢？连接实际上是通信双方交换控制信息，在套接字中记录这些必要信息并准备数据收发的一连串操作，在讲解具体的过程之前，我们先来说一说“连接”到底代表什么意思。

网线是一直连接着的，随时都有信号从中流过，如果通信过程只是将数据转换为电信号，那么这一操作随时都可以进行。不过，在这个时间点，也就是套接字刚刚创建完成时，当应用程序委托发送数据的时候，协议栈会如何操作呢？

套接字刚刚创建完成的时候，里面并没有存放任何数据，也不知道通信的对象是谁。在这个状态下，即便应用程序要求发送数据，协议栈也不知道数据应该发送给谁。浏览器可以根据网址来查询服务器的 IP 地址，而且根据规则也知道应该使用 80 号端口，但只有浏览器知道这些必要的信息是不够的，因为在调用 `socket` 创建套接字时，这些信息并没有传递给协议栈。因此，我们需要把服务器的 IP 地址和端口号等信息告知协议栈，这是连接操作的目的之一。

那么，服务器这边又是怎样的情况呢？服务器上也会创建套接字A，但服务器上的协议栈和客户端一样，只创建套接字是不知道应该和谁进行通信的。而且，和客户端不同的是，在服务器上，连应用程序也不知道通信对象是谁，这样下去永远也没法开始通信。于是，我们需要让客户端向服务器告知必要的信息，比如“我想和你开始通信，我的 IP 地址是 xxx.xxx.xxx.xxx，端口号是 yyyy。”可见，客户端向服务器传达开始通信的请求，也是连接操作的目的之一。

之前我们讲过，连接实际上是通信双方交换控制信息，在套接字中记录这些必要信息并准备数据收发的一连串操作，像上面提到的客户端将 IP 地址和端口号告知服务器这样的过程就属于交换控制信息的一个具体的例子。所谓控制信息，就是用来控制数据收发操作所需的一些信息，IP 地址和端口号就是典型的例子。除此之外还有其他一些控制信息，我们后面会逐一进行介绍。连接操作中所交换的控制信息是根据通信规则来确定的，只要根据规则执行连接操作，双方就可以得到必要的信息从而完成数据收发的准备。此外，当执行数据收发操作时，我们还需要一块用来临时存放要收发的数据的内存空间，这块内存空间称为缓冲区，它也是在连接操作的过程中分配的。上面这些就是“连接”这个词代表的具体含义。

负责保存控制信息的头部

关于控制信息，这里再补充一些。控制信息大体上分为两类。

第一类是客户端和服务端相互联络时交换的控制信息。这些信息不仅连接时需要，在数据收发和断开连接的整个通信过程中都需要，这些内容在 TCP 协议的规格中进行了定义。

这个表里面的这些字段就是 TCP 规格中定义的控制信息。这些字段是固定的，在连接、收发、断开等各个阶段中，每次客户端和服务端之间进行通信时，都需要提供这些控制信息。

<figure> </figure>

具体来说，这些信息会被添加在客户端与服务端之间传递的网络包的开头。在连接阶段，由于数据收发还没有开始，所以网络包中没有实际的数据，只有控制信息。

<figure> </figure>

这些控制信息位于网络包的开头，因此被称为头部。此外，以太网和 IP 协议也有自己的控制信息，这些信息也叫头部，为了避免各种不同的头部发生混淆，我们一般会记作 TCP 头部、以太网头部（MAC 头部）、IP 头部。

客户端和服务端在通信中会将必要的信息记录在头部并相互确认，例如下面这样。

- 发送方：“开始数据发送。”
- 接收方：“请继续。”
- 发送方：“现在发送的是 xx 号数据。”
- 接收方：“xx 号数据已收到。”
-（以下省略）

正是有了这样的交互过程，双方才能够进行通信。头部的信息非常重要，理解了头部各字段的含义，就等于理解了整个通信的过程。在后面介绍协议栈的工作过程时，我们将根据需要讲解头部各字段的含义，现在大家只要先记住头部是用来记录和交换控制信息的就可以了。

控制信息还有另外一类，那就是保存在套接字中，用来控制协议栈操作的信息。应用程序传递来的信息以及从通信对象接收到的信息都会保存在这里，还有收发数据操作的执行状态等信息也会保存

在这里，协议栈会根据这些信息来执行每一步的操作。我们可以说，套接字的控制信息和协议栈的程序本身其实是一体的，因此，“协议栈具体需要哪些信息”会根据协议栈本身的实现方式不同而不同，但这并没有什么问题。因为协议栈中的控制信息通信对方是看不见的，只要在通信时按照规则将必要的信息写入头部，客户端和服务端之间的通信就能够得以成立。例如，Windows 和 Linux 操作系统的内部结构不同，协议栈的实现方式不同，必要的控制信息也就不同。但即便如此，两种系统之间依然能够互相通信，同样地，计算机和手机之间也能够互相通信。正如前面所说，协议栈的实现不同，因此我们无法具体说明协议栈里到底保存了哪些控制信息，但可以用命令来显示一些重要的套接字控制信息（`netstat`），这些信息无论何种操作系统的协议栈都是共通的，通过理解这些重要信息，就能够理解协议栈的工作方式了。

连接操作的实际过程

了解了连接操作的含义，下面来看一下具体的操作过程。这个过程是从应用程序调用 Socket 库的 `connect()` 开始。

传入的参数有 IP 地址和端口号，这些参数会传递给协议栈中的 TCP 模块。然后，TCP 模块会与该 IP 地址对应的对象（服务器的 TCP 模块）交换控制信息，这一交互过程包括下面几个步骤。首先，客户端先创建一个包含表示开始数据收发操作的控制信息的头部。如表 2.1 所示，头部包含很多字段，这里要关注的重点是发送方和接收方的端口号。到这里，客户端（发送方）的套接字就准确找到了服务器（接收方）的套接字，也就是搞清楚了我应该连接哪个套接字。然后，我们将头部中的控制位的 SYN 比特设置为 1，大家可以认为它表示连接。此外还需要设置适当的序号和窗口大小，这一点我们会稍后详细讲解。

当 TCP 头部创建好之后，接下来 TCP 模块会将信息传递给 IP 模块并委托它进行发送。IP 模块执行网络包发送操作后，网络包就会通过网络到达服务器，然后服务器上的 IP 模块会将接收到的数据传递给 TCP 模块，服务器的 TCP 模块根据 TCP 头部中的信息找到端口号对应的套接字，也就是说，从处于等待连接状态的套接字中找到与 TCP 头部中记录的端口号相同的套接字就可以了。当找到对应的套接字之后，套接字中会写入相应的信息，并将状态改为正在连接。上述操作完成后，服务器的 TCP 模块会返回响应，这个过程和客户端一样，需要在 TCP 头部中设置发送方和接收方端口号以及 SYN 比特。此外，在返回响应时还需要将 ACK 控制位设为 1，这表示已经接收到相应的网络包。网络中经常会发生错误，网络包也会发生丢失，因此双方在通信时必须相互确认网络包是否已经送达，而设置 ACK 比特就是用来进行这一确认的。接下来，服务器 TCP 模块会将 TCP 头部传递给 IP 模块，并委托 IP 模块向客户端返回响应。然后，网络包就会返回到客户端，通过 IP 模块到达 TCP 模块，并通过 TCP 头部的信息确认连接服务器的操作是否成功。如果 SYN 为 1 则表示连接成功，这时会向套接字中写入服务器的 IP 地址、端口号等信息，同时还会将状态改为连接完毕。到这里，客户端的操作就已经完成，但其实还剩下最后一个步骤。刚才服务器返回响应时将 ACK 比特设置为 1，相应地，客户端也需要将 ACK 比特设置为 1 并发回服务器，告诉服务器刚才的响应包已经收到。当这个服务器收到这个返回包之后，连接操作才算全部完成。

现在，套接字就已经进入随时可以收发数据的状态了，大家可以认为这时有一根管子把两个套接字连接了起来。当然，实际上并不存在这么一根管子，不过这样想比较容易理解，网络业界也习惯这

样来描述。这根管子，我们称之为连接(Connection)。只要数据传输过程在持续，也就是在调用 `close()` 断开之前，连接是一直存在的。

建立连接之后，协议栈的连接操作就结束了，也就是说 `connect()` 已经执行完毕，控制流程被交回到应用程序。

收发数据

将 HTTP 请求消息交给协议栈

当控制流程从 `connect()` 回到应用程序之后，接下来就进入数据收发阶段了。数据收发操作是从应用程序调用 `write()` 将要发送的数据交给协议栈开始的，协议栈收到数据后执行发送操作，这一操作包含如下要点。

首先，协议栈并不关心应用程序传来的数据是什么内容。应用程序在调用 `write()` 时会指定发送数据的长度，在协议栈看来，要发送的数据就是一定长度的二进制字节序列而已。

其次，协议栈并不是一收到数据就马上发送出去，而是会将数据存放在内部的发送缓冲区中，并等待应用程序的下一段数据。这样做是有道理的。应用程序交给协议栈发送的数据长度是由应用程序本身来决定的，不同的应用程序在实现上有所不同，有些程序会一次性传递所有的数据，有些程序则会逐字节或者逐行传递数据。总之，一次将多少数据交给协议栈是由应用程序自行决定的，协议栈并不能控制这一行为。在这样的情况下，如果一收到数据就马上发送出去，就可能会发送大量的小包，导致网络效率下降，因此需要在数据积累到一定量时再发送出去。至于要积累多少数据才能发送，不同种类和版本的操作系统会有所不同，不能一概而论，但都是根据下面几个要素来判断的。

第一个判断要素是每个网络包能容纳的数据长度，协议栈会根据一个叫作 MTU 的参数来进行判断。MTU 表示一个网络包的最大长度，在以太网中一般是 1500 字节。MTU 是包含头部的总长度，因此需要从 MTU 减去头部的长度，然后得到的长度就是一个网络包中所能容纳的最大数据长度，这一长度叫作 MSS。当从应用程序收到的数据长度超过或者接近 MSS 时再发送出去，就可以避免发送大量小包的问题了。

<figure> </figure>

另一个判断要素是时间。当应用程序发送数据的频率不高的时候，如果每次都等到长度接近 MSS 时再发送，可能会因为等待时间太长而造成发送延迟，这种情况下，即便缓冲区中的数据长度没有达到 MSS，也应该果断发送出去。为此，协议栈的内部有一个计时器，当经过一定时间之后，就会把网络包发送出去。

判断要素就是这两个，但它们其实是互相矛盾的。如果长度优先，那么网络的效率会提高，但可能会因为等待填满缓冲区而产生延迟；相反地，如果时间优先，那么延迟时间会变少，但又会降低网络的效率。因此，在进行发送操作时需要综合考虑这两个要素以达到平衡。不过，TCP 协议规格中

并没有告诉我们怎样才能平衡，因此实际如何判断是由协议栈的开发者来决定的，也正是由于这个原因，不同种类和版本的操作系统在相关操作上也就存在差异。

正如前面所说，如果仅靠协议栈来判断发送的时机可能会带来一些问题，因此协议栈也给应用程序保留了控制发送时机的余地。应用程序在发送数据时可以指定一些选项，比如如果指定“不等待填满缓冲区直接发送”，则协议栈就会按照要求直接发送数据。像浏览器这种会话型的应用程序在向服务器发送数据时，等待填满缓冲区导致延迟会产生很大影响，因此一般会使用直接发送的选项。

对较大的数据进行拆分

HTTP 请求消息一般不会很长，一个网络包就能装得下，但如果其中要提交表单数据，长度就可能超过一个网络包所能容纳的数据量，比如在博客或者论坛上发表一篇长文就属于这种情况。

这种情况下，发送缓冲区中的数据就会超过 MSS 的长度，这时我们当然不需要继续等待后面的数据了。发送缓冲区中的数据会被以 MSS 长度为单位进行拆分，拆分出来的每块数据会被放进单独的网络包中。

根据发送缓冲区中的数据拆分的情况，当判断需要发送这些数据时，就在每一块数据前面加上 TCP 头部，并根据套接字中记录的控制信息标记发送方和接收方的端口号，然后交给 IP 模块来执行发送数据的操作。

<figure> </figure>

使用 ACK 号确认网络包收到

到这里，网络包已经装好数据并发往服务器了，但数据发送操作还没有结束。TCP 具备确认对方是否成功收到网络包，以及当对方没收到时进行重发的功能，因此在发送网络包之后，接下来还需要进行确认操作。

<figure> </figure>

首先，TCP 模块在拆分数据时，会先算好每一块数据相当于从头开始的第几个字节，接下来在发送这一块数据时，将算好的字节数写在 TCP 头部中，“序号”字段就是派在这个用场上的。

然后，发送数据的长度也需要告知接收方，不过这个并不是放在 TCP 头部里面的，因为用整个网络包的长度减去头部的长度就可以得到数据的长度，所以接收方可以用这种方法来进行计算。有了上面两个数值，我们就可以知道发送的数据是从第几个字节开始，长度是多少了。

通过这些信息，接收方还能够检查收到的网络包有没有遗漏。例如，假设上次接收到第 1460 字节，那么接下来如果收到序号为 1461 的包，说明中间没有遗漏；但如果收到的包序号为 2921，那就说明中间有包遗漏了。像这样，如果确认没有遗漏，接收方会将到目前为止接收到的数据长度加起来，计算出一共已经收到了多少个字节，然后将这个数值写入 TCP 头部的 ACK 号中发送给发送方。简单来说，发送方说的是“现在发送的是从第 ×× 字节开始的部分，一共有 ×× 字节哦！”而接收方则回复说，“到第 ×× 字节之前的数据我已经都收到了哦！”这个返回 ACK 号的操作被称为确认响应，通过这样的方式，发送方就能够确认对方到底收到了多少数据。

上图的例子和实际情况还是有些出入的。在实际的通信中，序号并不是从 1 开始的，而是需要用随机数计算出一个初始值，这是因为如果序号都从 1 开始，通信过程就会非常容易预测，有人会利用这一点来发动攻击。但是如果初始值是随机的，那么对方就搞不清楚序号到底是从多少开始计算的，因此需要在开始收发数据之前将初始值告知通信对象。大家应该还记得在我们刚才讲过的连接过程中，有一个将 SYN 控制位设为 1 并发送给服务器的操作，就是在这一步将序号的初始值告知对方的。实际上，在将 SYN 设为 1 的同时，还需要同时设置序号字段的值，而这里的值就代表序号的初始值。

前面介绍了通过序号和 ACK 号来进行数据确认的思路，但仅凭这些还不够，因为我们刚刚只考虑了单向的数据传输，但 TCP 数据收发是双向的，在客户端向服务器发送数据的同时，服务器也会向客户端发送数据，因此必须要想办法应对这样的情况。不过，这其实也不难，上图中展示的客户端向服务器发送数据的情形，我们只要增加一种左右相反的情形就可以了，如下图所示。

<figure> </figure>

首先客户端先计算出一个序号，然后将序号和数据一起发送给服务器，服务器收到之后会计算 ACK 号并返回给客户端；相反地，服务器也需要先计算出另一个序号，然后将序号和数据一起发送给客户端，客户端收到之后计算 ACK 号并返回给服务器。此外，客户端和服务器双方都需要各自计算序号，因此双方需要在连接过程中互相告知自己计算的序号初始值。

明白原理之后我们来看一下实际的工作过程。

<figure> </figure>

首先，客户端在连接时需要计算出与从客户端到服务器方向通信相关的序号初始值，并将这个值发送给服务器①。接下来，服务器会通过这个初始值计算出 ACK 号并返回给客户端②。初始值有可能在通信过程中丢失，因此当服务器收到初始值后需要返回 ACK 号作为确认。同时，服务器也需要计算出与从服务器到客户端方向通信相关的序号初始值，并将这个值发送给客户端③。接下来像刚才一样，客户端也需要根据服务器发来的初始值计算出 ACK 号并返回给服务器④。到这里，序号和 ACK 号都已经准备完成了，接下来就可以进入数据收发阶段了。数据收发操作本身是可以双向同时进行的，但 Web 中是先由客户端向服务器发送请求，序号也会跟随数据一起发送④。然后，服务器收到数据后再返回 ACK 号⑤。从服务器向客户端发送数据的过程则正好相反⑥⑦。

TCP 采用这样的方式确认对方是否收到了数据，在得到对方确认之前，发送过的包都会保存在发送缓冲区中。如果对方没有返回某些包对应的 ACK 号，那么就重新发送这些包。

这一机制非常强大。通过这一机制，我们可以确认接收方有没有收到某个包，如果没有收到则重新发送，这样一来，无论网络中发生任何错误，我们都可以发现并采取补救措施（重传网络包）。反过来说，有了这一机制，我们就不需要在其他地方对错误进行补救了。

因此，网卡、集线器、路由器都没有错误补偿机制，一旦检测到错误就直接丢弃相应的包。应用程序也是一样，因为采用 TCP 传输，即便发生一些错误对方最终也能够收到正确的数据，所以应用程序只管自顾自地发送这些数据就好了。不过，如果发生网络中断、服务器宕机等问题，那么无论

TCP 怎样重传都不管用。这种情况下，无论如何尝试都是徒劳，因此 TCP 会在尝试几次重传无效之后强制结束通信，并向应用程序报错。

根据网络包平均往返时间调整 ACK 号等待时间

前面说的只是一些基本原理，实际上网络的错误检测和补偿机制非常复杂。下面来说几个关键点，首先是返回 ACK 号的等待时间（这个等待时间叫超时时间）。

当网络传输繁忙时就会发生拥塞，ACK 号的返回会变慢，这时我们就必须将等待时间设置得稍微长一点，否则可能会发生已经重传了包之后，前面的 ACK 号才姗姗来迟的情况。这样的重传是多余的，看上去只是多发一个包而已，但它造成的后果却没那么简单。因为 ACK 号的返回变慢大多是由于网络拥塞引起的，因此如果此时再出现很多多余的重传，对于本来就很拥塞的网络来说无疑是雪上加霜。那么等待时间是不是越长越好呢？也不是。如果等待时间过长，那么包的重传就会出现很大的延迟，也会导致网络速度变慢。

看来等待时间需要设为一个合适的值，不能太长也不能太短，但这谈何容易。根据服务器物理距离的远近，ACK 号的返回时间也会产生很大的波动，而且我们还必须考虑到拥塞带来的影响。例如，在公司里的局域网环境下，几毫秒就可以返回 ACK 号，但在互联网环境中，当遇到拥塞时需要几百毫秒才能返回 ACK 号也并不稀奇。

正因为波动如此之大，所以将等待时间设置为一个固定值并不是一个好办法。因此，TCP 采用了动态调整等待时间的方法，这个等待时间是根据 ACK 号返回所需的时间来判断的。具体来说，TCP 会在发送数据的过程中持续测量 ACK 号的返回时间，如果 ACK 号返回变慢，则相应延长等待时间；相对地，如果 ACK 号马上就能返回，则相应缩短等待时间。

使用窗口有效管理 ACK 号

每发送一个包就等待一个 ACK 号的方式是最简单也最容易理解的，但在等待 ACK 号的这段时间中，如果什么都不做那实在太浪费了。

<figure> </figure>

为了减少这样的浪费，TCP 采用上图（b）这样的滑动窗口方式来管理数据发送和 ACK 号的操作。所谓滑动窗口，就是在发送一个包之后，不等待 ACK 号返回，而是直接发送后续的一系列包。这样一来，等待 ACK 号的这段时间就被有效利用起来了。

虽然这样做能够减少等待 ACK 号时的时间浪费，但有一些问题需要注意。在一来一回方式中，接收方完成接收操作后返回 ACK 号，然后发送方收到 ACK 号之后才继续发送下一个包，因此不会出现发送的包太多接收方处理不过来的情况。但如果不等返回 ACK 号就连续发送包，就有可能出现发送包的频率超过接收方处理能力的情况。

下面来具体解释一下。当接收方的 TCP 收到包后，会先将数据存放到接收缓冲区中。然后，接收方需要计算 ACK 号，将数据块组装起来还原成原本的数据并传递给应用程序，如果这些操作还没完成下一个包就到了也不用担心，因为下一个包也会被暂存在接收缓冲区中。如果数据到达的速率比处

理这些数据并传递给应用程序的速率还要快，那么接收缓冲区中的数据就会越堆越多，最后就会溢出。缓冲区溢出之后，后面的数据就进不来了，因此接收方就收不到后面的包了，这就和中途出错的结果是一样的，也就意味着超出了接收方处理能力。我们可以通过下面的方法来避免这种情况的发生。首先，接收方需要告诉发送方自己最多能接收多少数据，然后发送方根据这个值对数据发送操作进行控制，这就是滑动窗口方式的基本思路。

关于滑动窗口的具体工作方式，还是看图更容易理解。

<figure> </figure>

在这张图中，接收方将数据暂存到接收缓冲区中并执行接收操作。当接收操作完成后，接收缓冲区中的空间会被释放出来，也就可以接收更多的数据了，这时接收方会通过 TCP 头部中的窗口字段将自己能接收的数据量告知发送方。这样一来，发送方就不会发送过多的数据，导致超出接收方的处理能力了。

此外，单从图上看，大家可能会以为接收方在等待接收缓冲区被填满之前似乎什么都没做，实际上并不是这样。这张图是为了讲解方便，故意体现一种接收方来不及处理收到的包，导致缓冲区被填满的情况。实际上，接收方在收到数据之后马上就会开始进行处理，如果接收方的性能高，处理速度比包的到达速率还快，缓冲区马上就会被清空，并通过窗口字段告知发送方。

还有，图中只显示了从右往左发送数据的操作，实际上和序号、ACK 号一样，发送操作也是双向进行的。前面提到的能够接收的最大数据量称为窗口大小，它是 TCP 调优参数中非常有名的一个。

ACK 与窗口的合并

要提高收发数据的效率，还需要考虑另一个问题，那就是返回 ACK 号和更新窗口的时机。如果假定这两个参数是相互独立的，分别用两个单独的包来发送，结果会如何呢？

首先，什么时候需要更新窗口大小呢？当收到的数据刚刚开始填入缓冲区时，其实没必要每次都向发送方更新窗口大小，因为只要发送方在每次发送数据时减掉已发送的数据长度就可以自行计算出当前窗口的剩余长度。因此，更新窗口大小的时机应该是接收方从缓冲区中取出数据传递给应用程序的时候。这个操作是接收方应用程序发出请求时才会进行的，而发送方不知道什么时候会进行这样的操作，因此当接收方将数据传递给应用程序，导致接收缓冲区剩余容量增加时，就需要告知发送方，这就是更新窗口大小的时机。

那么 ACK 号又是什么情况呢？当接收方收到数据时，如果确认内容没有问题，就应该向发送方返回 ACK 号，因此我们可以认为收到数据之后马上就应该进行这一操作。

如果将前面两个因素结合起来看，首先，发送方的数据到达接收方，在接收操作完成之后就需要向发送方返回 ACK 号，而再经过一段时间，当数据传递给应用程序之后才需要更新窗口大小。但如果根据这样的设计来实现，每收到一个包，就需要向发送方分别发送 ACK 号和窗口更新这两个单独的包。这样一来，接收方发给发送方的包就太多了，导致网络效率下降。

因此，接收方在发送 ACK 号和窗口更新时，并不会马上把包发送出去，而是会等待一段时间，在这个过程中很有可能会出现其他的通知操作，这样就可以把两种通知合并在一个包里面发送了。举个

例子，在等待发送 ACK 号的时候正好需要更新窗口，这时就可以把 ACK 号和窗口更新放在一个包里发送，从而减少包的数量。当需要连续发送多个 ACK 号时，也可以减少包的数量，这是因为 ACK 号表示的是已收到的数据量，也就是说，它是告诉发送方目前已接收的数据的最后位置在哪里，因此当需要连续发送 ACK 号时，只要发送最后一个 ACK 号就可以了，中间的可以全部省略。当需要连续发送多个窗口更新时也可以减少包的数量，因为连续发生窗口更新说明应用程序连续请求了数据，接收缓冲区的剩余空间连续增加。这种情况和 ACK 号一样，可以省略中间过程，只要发送最终的结果就可以了。

接收 HTTP 响应消息

到这里，我们已经讲解完协议栈接到浏览器的委托后发送 HTTP 请求消息的一系列操作过程了。

不过，浏览器的工作并非到此为止。发送 HTTP 请求消息后，接下来还需要等待 Web 服务器返回响应消息。对于响应消息，浏览器需要进行接收操作，这一操作也需要协议栈的参与。按照探索之旅的思路，本来是应该按照访问 Web 服务器的顺序逐一讲解其中的每一步操作，也就是说接收 HTTP 响应消息应该放在最后再讲，但这样一来大家可能容易忘记前面的部分，所以我们就把这部分内容放在这里讲一讲。

首先，浏览器在委托协议栈发送请求消息之后，会调用 read 程序来获取响应消息。然后，控制流程会通过 read 转移到协议栈，然后协议栈会执行接下来的操作。和发送数据一样，接收数据也需要将数据暂存到接收缓冲区中，这里的操作过程如下。首先，协议栈尝试从接收缓冲区中取出数据并传递给应用程序，但这个时候请求消息刚刚发送出去，响应消息可能还没返回。响应消息的返回还需要等待一段时间，因此这时接收缓冲区中并没有数据，那么接收数据的操作也就无法继续。这时，协议栈会将应用程序的委托，也就是从接收缓冲区中取出数据并传递给应用程序的工作暂时挂起，等服务器返回的响应消息到达之后再继续执行接收操作。

协议栈接收数据的具体操作过程已经在发送数据的部分讲解过了，因此这里我们就简单总结一下。首先，协议栈会检查收到的数据块和 TCP 头部的内容，判断是否有数据丢失，如果没有问题则返回 ACK 号。然后，协议栈将数据块暂存到接收缓冲区中，并将数据块按顺序连接起来还原出原始的数据，最后将数据交给应用程序。具体来说，协议栈会将接收到的数据复制到应用程序指定的内存地址中，然后将控制流程交回应用程序。将数据交给应用程序之后，协议栈还需要找到合适的时机向发送方发送窗口更新。

删除套接字

数据发送完毕后断开连接

既然我们已经讲解到了这里，那么索性把数据收发完成后协议栈要执行的操作也讲一讲吧。这样一来，从创建套接字到连接、收发数据、断开连接、删除套接字这一系列关于收发数据的操作就全部讲完了。

毫无疑问，收发数据结束的时间点应该是应用程序判断所有数据都已经发送完毕的时候。这时，数据发送完毕的一方会发起断开过程，但不同的应用程序会选择不同的断开时机。以 Web 为例，浏览器向 Web 服务器发送请求消息，Web 服务器再返回响应消息，这时收发数据的过程就全部结束了，服务器一方会发起断开过程。当然，可能也有一些程序是客户端发送完数据就结束了，不用等服务器响应，这时客户端会先发起断开过程。这一判断是应用程序作出的，协议栈在设计上允许任何一方先发起断开过程。

无论哪种情况，完成数据发送的一方会发起断开过程，这里我们以服务器一方发起断开过程为例来进行讲解。首先，服务器一方的应用程序会调用 Socket 库的 `close()` 程序。然后，服务器的协议栈会生成包含断开信息的 TCP 头部，具体来说就是将控制位中的 FIN 比特设为 1。接下来，协议栈会委托 IP 模块向客户端发送数据。同时，服务器的套接字中也会记录下断开操作的相关信息。

<figure> </figure>

客户端收到服务器发来的 FIN 为 1 的 TCP 头部时，客户端的协议栈会将套接字标记为进入断开操作状态。然后，为了告知服务器已收到 FIN 为 1 的包，客户端会向服务器返回一个 ACK 号②。这些操作完成后，协议栈就可以等待应用程序来取数据了。

过了一会儿，应用程序就会调用 `read()` 来读取数据。这时，协议栈不会向应用程序传递数据，而是会告知应用程序（浏览器）来自服务器的数据已经全部收到了。根据规则，服务器返回请求之后，Web 通信操作就全部结束了，因此只要收到服务器返回的所有数据，客户端的操作也就随之结束了。因此，客户端应用程序会调用 `close()` 来结束数据收发操作，这时客户端的协议栈也会和服务器一样，生成一个 FIN 比特为 1 的 TCP 包，然后委托 IP 模块发送给服务器③。一段时间之后，服务器就会返回 ACK 号④。到这里，客户端和服务器的通信就全部结束了。

删除套接字

和服务器的通信结束之后，用来通信的套接字也就不会再使用了，这时我们就可以删除这个套接字了。不过，套接字并不会立即被删除，而是会等待一段时间之后再被删除。

等待这段时间是为了防止误操作，引发误操作的原因有很多，这里无法全部列举，下面来举一个最容易理解的例子。客户端先发起断开，则断开的操作顺序如下。

- （1）客户端发送 FIN
- （2）服务器返回 ACK 号
- （3）服务器发送 FIN
- （4）客户端返回 ACK 号

如果最后客户端返回的 ACK 号丢失了，结果会如何呢？这时，服务器没有接收到 ACK 号，可能会重发一次 FIN。如果这时客户端的套接字已经删除了，会发生什么事呢？套接字被删除，那么套接字中保存的控制信息也就跟着消失了，套接字对应的端口号就会被释放出来。这时，如果别的应用程序要创建套接字，新套接字碰巧又被分配了同一个端口号，而服务器重发的 FIN 正好到达，会怎么样呢？本来这个 FIN 是要发给刚刚删除的那个套接字的，但新套接字具有相同的端口号，于是这个

FIN 就会错误地跑到新套接字里面，新套接字就开始执行断开操作了。之所以不马上删除套接字，就是为了防止这样的误操作。

至于具体等待多长时间，这和包重传的操作方式有关。网络包丢失之后会进行重传，这个操作通常要持续几分钟。如果重传了几分钟之后依然无效，则停止重传。在这段时间内，网络中可能存在重传的包，也就有可能发生前面讲到的这种误操作，因此需要等待到重传完全结束。协议中对于这个等待时间没有明确的规定，一般来说会等待几分钟之后再删除套接字。

收发小结

到这里，用 TCP 协议收发应用程序数据的操作就全部结束了。这部分内容的讲解比较长，所以最后我们再整理一下。

<figure> </figure>

数据收发操作的第一步是创建套接字。一般来说，服务器一方的应用程序在启动时就会创建好套接字并进入等待连接的状态。客户端则一般是在用户触发特定动作，需要访问服务器的时候创建套接字。在这个阶段，还没有开始传输网络包。

创建套接字之后，客户端会向服务器发起连接操作。首先，客户端会生成一个 SYN 为 1 的 TCP 包并发送给服务器。这个 TCP 包的头部还包含了客户端向服务器发送数据时使用的初始序号，以及服务器向客户端发送数据时需要用到的窗口大小。当这个包到达服务器之后，服务器会返回一个 SYN 为 1 的 TCP 包，这个包的头部中也包含了序号和窗口大小，此外还包含表示确认已收到包①的 ACK 号。当这个包到达客户端时，客户端会向服务器返回一个包含表示确认的 ACK 号的 TCP 包。到这里，连接操作就完成了，双方进入数据收发阶段。

数据收发阶段的操作根据应用程序的不同而有一些差异，以 Web 为例，首先客户端会向服务器发送请求消息。TCP 会将请求消息切分成一定大小的块，并在每一块前面加上 TCP 头部，然后发送给服务器。TCP 头部中包含序号，它表示当前发送的是第几个字节的数据。当服务器收到数据时，会向客户端返回 ACK 号。在最初的阶段，服务器只是不断接收数据，随着数据收发的进行，数据不断传递给应用程序，接收缓冲区就会被逐步释放。这时，服务器需要将新的窗口大小告知客户端。当服务器收到客户端的请求消息后，会向客户端返回响应消息，这个过程和刚才的过程正好相反。

服务器的响应消息发送完毕之后，数据收发操作就结束了，这时就会开始执行断开操作。以 Web 为例，服务器会先发起断开过程。在这个过程中，服务器先发送一个 FIN 为 1 的 TCP 包，然后客户端返回一个表示确认收到的 ACK 号。接下来，双方还会交换一组方向相反的 FIN 为 1 的 TCP 包和包含 ACK 号的 TCP 包。最后，在等待一段时间后，套接字会被删除。

IP与以太网的包收发

包的基本知识

TCP 模块在执行连接、收发、断开等各阶段操作时，都需要委托 IP 模块将数据封装成包发送给通信对象。我们在 TCP 的讲解中也经常提到 IP，下面就来讨论一下 IP 模块是如何将包发送给对方的。

正式开始这个话题之前，我们先来介绍一下关于网络包的一些基本知识。

首先，包是由头部和数据两部分构成的。头部包含目的地址等控制信息，快递单；头部后面就是委托方要发送给对方的数据，快递包裹里的东西。

<figure> </figure>

一个包发往目的地的过程如下图。

<figure> </figure>

首先，发送方的网络设备会负责创建包，创建包的过程就是生成含有正确控制信息的头部，然后再附加上要发送的数据。接下来，包会被发往最近的网络转发设备。当到达最近的转发设备之后，转发设备会根据头部中的信息判断接下来应该发往哪里。这个过程需要用到一张表，这张表里面记录了每一个地址对应的发送方向，也就是按照头部里记录的地址在表里进行查询，并根据查到的信息判断接下来应该发往哪个方向。比如，如果查表的结果是“目标地址为 ×××× 的包应该发到 ×××× 号线路”，那么转发设备就会把这个包发到 ×××× 号线路去。接下来，包在向目的地移动的过程中，又会到达下一个转发设备，然后又会按照同样的方式被发往下一个转发设备。就这样，经过多个转发设备的接力之后，包最终就会到达接收方的网络设备。当然，发送方向接收方发送一个包，接收方可能也会向发送方返回一个包，此时的发送方到了接下来的某个时刻就会变成接收方。因此，我们不需要把发送方和接收方明确区分开来，在这里我们把发送方和接收方统称为终端节点。

前面介绍的这些基本知识，对于各种通信方式都是适用的，当然也适用于 TCP/IP 网络。不过，TCP/IP 包的结构是在这个基本结构的基础上扩展出来的，因此更加复杂。前面，我们讲过子网的概念，还讲过网络中有路由器和集线器两种不同的转发设备，它们在传输网络包时有着各自的分工。

- 路由器根据目标地址判断下一个路由器的位置
- 集线器在子网中将网络包传输到下一个路由

实际上，集线器是按照以太网规则传输包的设备，而路由器是按照 IP 规则传输包的设备，因此我们也可以作如下理解。

- IP 协议根据目标地址判断下一个 IP 转发设备的位置
- 子网中的以太网协议将包传输到下一个转发设备

具体来说，TCP/IP 包包含如下两个头部。

- MAC 头部（用于以太网协议）
- IP 头部（用于 IP 协议）

这两个头部分别具有不同的作用。首先，发送方将包的目的地，也就是要访问的服务器的 IP 地址写入 IP 头部中。这样一来，我们就知道这个包应该发往哪里，IP 协议就可以根据这一地址查找包的传

输方向，从而找到下一个路由器的位置，也就是图 2.16 中的路由器 R1。接下来，IP 协议会委托以太网协议将包传输过去。这时，IP 协议会查找下一个路由器的以太网地址（MAC 地址），并将这个地址写入 MAC 头部中。这样一来，以太网协议就知道要将这个包发到哪一个路由器上了。

网络包在传输过程中会经过集线器，集线器是根据以太网协议工作的设备。为了判断包接下来应该向什么地方传输，集线器里有一张表（用于以太网协议的表），可根据以太网头部中记录的目的地信息查出相应的传输方向。这张图中只有一个集线器，当存在多个集线器时，网络包会按顺序逐一通过这些集线器进行传输。接下来，包会到达下一个路由器。路由器中有一张 IP 协议的表，可根据这张表以及 IP 头部中记录的目的地信息查出接下来应该发往哪个路由器。为了将包发到下一个路由器，我们还需要查出下一个路由器的 MAC 地址，并记录到 MAC 头部中，大家可以理解为改写了 MAC 头部。这样，网络包就又被发往下一个节点了。

再往后的过程图上就没有画出来了。网络包会通过路由器到达下一个路由器 R2。这个过程不断重复，最终网络包就会被送到目的地，当目的地设备成功接收之后，网络包的传输过程就结束了。

前面介绍的就是在 TCP/IP 网络中，一个网络包从出发到到达目的地的全过程。虽然看起来有点复杂，不过设计这样的分工是有原因的。前面讲了 IP 和以太网的分工，其中以太网的部分也可以替换成其他的东西，例如无线局域网、ADSL、FTTH 等，它们都可以替代以太网的角色帮助 IP 协议来传输网络包。因此，将 IP 和负责传输的网络分开，可以更好地根据需要使用各种通信技术。像互联网这样庞大复杂的网络，在架构上需要保证灵活性，这就是设计这种分工方式的原因。

包收发操作

了解了整体流程之后，下面来讲一讲在协议栈中 IP 模块是如何完成包收发操作的。尽管我们说 IP 模块负责将包发给对方，但实际上将包从发送方传输到接收方的工作是由集线器、路由器等网络设备来完成的，因此 IP 模块仅仅是整个包传输过程的入口而已。即便如此，IP 模块还是有很多工作需要完成，首先我们先粗略地整理一下。

<figure> </figure>

包收发操作的起点是 TCP 模块委托 IP 模块发送包的操作（上图“①发送”）。这个委托的过程就是 TCP 模块在数据块的前面加上 TCP 头部，然后整个传递给 IP 模块，这部分就是网络包的内容。与此同时，TCP 模块还需要指定通信对象的 IP 地址，也就是需要写清楚“将什么内容发给谁”。

收到委托后，IP 模块会将包的内容当作一整块数据，在前面加上包含控制信息的头部。刚才我们讲过，IP 模块会添加 IP 头部和 MAC 头部这两种头部。IP 头部中包含 IP 协议规定的、根据 IP 地址将包发往目的地所需的控制信息；MAC 头部包含通过以太网的局域网将包传输至最近的路由器所需的控制信息。关于 IP 头部和 MAC 头部的区别以及其中包含的控制信息的含义，我们将稍后介绍。总之，加上这两个头部之后，一个包就封装好了，这些就是 IP 模块负责的工作。

接下来，封装好的包会被交给网络硬件（“②发送”），例如以太网、无线局域网等。网络硬件可能是插在计算机主板上的板卡，也可能是笔记本电脑上的 PCMCIA 卡，或者是计算机主板上集成的芯片，不同形态的硬件名字也不一样，本书将它们统称为网卡。传递给网卡的网络包是由一连串 0 和 1

组成的数字信息，网卡会将这些数字信息转换为电信号或光信号，并通过网线（或光纤）发送出去，然后这些信号就会到达集线器、路由器等转发设备，再由转发设备一步一步地送达接收方。

包送达对方之后，对方会作出响应。返回的包也会通过转发设备发送回来，然后我们需要接收这个包。接收的过程和发送的过程是相反的，信息先以电信号的形式从网线传输进来，然后由网卡将其转换为数字信息并传递给 IP 模块（“③接收”）。接下来，IP 模块会将 MAC 头部和 IP 头部后面的内容，也就是 TCP 头部加上数据块，传递给 TCP 模块。接下来的操作就是我们之前讲过的 TCP 模块负责的部分了。

生成包含接收方 IP 地址的 IP 头部

下面来看一看 IP 模块的具体工作过程。IP 模块接受 TCP 模块的委托负责包的收发工作，它会生成 IP 头部并附加在 TCP 头部前面。IP 头部包含的内容下表所示

<figure> </figure>

其中最重要的内容就是 IP 地址，它表示这个包应该发到哪里去。这个地址是由 TCP 模块告知的，而 TCP 又是在执行连接操作时从应用程序那里获得这个地址的，因此这个地址的最初来源就是应用程序。IP 不会自行判断包的目的地，而是将包发往应用程序指定的接收方，即便应用程序指定了错误的 IP 地址，IP 模块也只能照做。当然，这样做肯定会出错，但这个责任应该由应用程序来承担。

头部中还需要填写发送方的 IP 地址，大家可以认为是发送方计算机的 IP 地址，实际上“计算机的 IP 地址”这种说法并不准确。一般的客户端计算机上只有一块网卡，因此也就只有一个 IP 地址，这种情况下我们可以认为这个 IP 地址就是计算机的 IP 地址，但如果计算机上有多个网卡，情况就没那么简单了。IP 地址实际上并不是分配给计算机的，而是分配给网卡的，因此当计算机上存在多块网卡时，每一块网卡都会有自己的 IP 地址。很多服务器上都会安装多块网卡，这时一台计算机就有多个 IP 地址，在填写发送方 IP 地址时就需要判断到底应该填写哪个地址。这个判断相当于在多块网卡中判断应该使用哪一块网卡来发送这个包，也就相当于判断应该把包发往哪个路由器，因此只要确定了目标路由器，也就确定了应该使用哪块网卡，也就确定了发送方的 IP 地址。

那么，我们应该如何判断应该把包交给哪块网卡呢？其实和路由器使用 IP 表判断下一个路由器位置的操作是一样的。因为协议栈的 IP 模块与路由器中负责包收发的部分都是根据 IP 协议规则来进行包收发操作的，所以它们也都用相同的方法来判断把包发送给谁。

这个“IP 表”叫作路由表，我们将在第 3 章探索路由器时详细介绍它的用法，这里先简单讲个大概。在 windows 的 cmd 里，输入 `route print` 命令显示路由表。如图所示，

<figure> </figure>

首先，我们对套接字中记录的目的地 IP 地址与路由表左侧的 Network Destination 栏进行比较，找到对应的一行。例如，TCP 模块告知的目标 IP 地址为 192.168.1.21，那么就对应图中的第 6 行，因为它和 192.168.1 的部分相匹配。如果目标 IP 地址为 10.10.1.166，那么就和 10.10.1 的部分相匹配，所以对应第 3 行。以此类推，我们需要找到与 IP 地址左边部分相匹配的条目，找到相应的条目之后，接下来看从右边数第 2 列和第 3 列的内容。右起第 2 列，也就是 Interface 列，表示网卡等网络接口，这

些网络接口可以将包发送给通信对。此外，右起第 3 列，即 **Gateway** 列表示下一个路由器的 IP 地址，将包发给这个 IP 地址，该地址对应的路由器就会将包转发到目标地址。路由表的第 1 行中，目标地址和子网掩码都是 0.0.0.0，这表示默认网关，如果其他所有条目都无法匹配，就会自动匹配这一行。

这样一来，我们就可以判断出应该使用哪块网卡来发送包了，然后就可以在 IP 头部的发送方 IP 地址中填上这块网卡对应的 IP 地址。

接下来还需要填写协议号，它表示包的内容是来自哪个模块的。例如，如果是 TCP 模块委托的内容，则设置为 06（十六进制），如果是 UDP 模块委托的内容，则设置为 17（十六进制），这些值都是按照规则来设置的。在现在我们使用的浏览器中，HTTP 请求消息都是通过 TCP 来传输的，因此这里就会填写表示 TCP 的 06（十六进制）。

其他字段内也需要填写相应的值，但对大局没什么影响，我们会在第 3 章进行介绍，这里就先省略了。

生成以太网用的 MAC 头部

生成了 IP 头部之后，接下来 IP 模块还需要在 IP 头部的前面加上 MAC 头部。

<figure> </figure>

IP 头部中的接收方 IP 地址表示网络包的目的地，通过这个地址我们就可以判断要将包发到哪里，但在以太网的世界中，TCP/IP 的这个思路是行不通的。以太网在判断网络包目的地时和 TCP/IP 的方式不同，因此必须采用相匹配的方式才能在以太网中将包发往目的地，而 MAC 头部就是干这个用的。

IP 模块在生成 IP 头部之后，会在它前面再加上 MAC 头部。MAC 头部是以太网使用的头部，它包含了接收方和发送方的 MAC 地址等信息。关于以太网的结构我们稍后会进行介绍，但下面的内容需要一些 MAC 头部的相关知识才能理解，因此先介绍一些最基础的。MAC 头部的开头是接收方和发送方的 MAC 地址，大家可以认为它们和 IP 头部中的接收方和发送方 IP 地址的功能差不多，只不过 IP 地址的长度为 32 比特，而 MAC 地址为 48 比特。此外，IP 地址是类似多少弄多少号这种现实中地址的层次化的结构，而 MAC 地址中的 48 比特可以看作是一个整体。尽管有上述差异，但从表示接收方和发送方的意义上来说，MAC 地址和 IP 地址是没有区别的，因此大家可以暂且先把它当成是一回事。第 3 个以太类型字段和 IP 头部中的协议号类似。在 IP 中，协议号表示 IP 头部后面的包内容的类型；而在以太网中，我们可以认为以太网类型后面就是以太网包的内容，而以太类型就表示后面内容的类型。以太网包的内容可以是 IP、ARP 等协议的包，它们都有对应的值，这也是根据规则来确定的。

生成 MAC 头部时，只要设置上表中的 3 个字段就可以了。方便起见，我们按照从下往上的顺序来对表进行讲解。首先是“以太类型”，这里填写表示 IP 协议的值 0800（十六进制）。接下来是发送方 MAC 地址，这里填写网卡本身的 MAC 地址。MAC 地址是在网卡生产时写入 ROM 里的，只要将这个值读取出来写入 MAC 头部就可以了。对于多块网卡的情况，请大家回想一下设置发送方 IP 地址

的方法。设置发送方 IP 地址时，我们已经判断出了从哪块网卡发送这个包，那么现在只要将这块网卡对应的 MAC 地址填进去就好了。

前面这些还比较简单，而接收方 MAC 地址就有点复杂了。只要告诉以太网对方的 MAC 的地址，以太网就会帮我们包发送过去，那么很显然这里应该填写对方的 MAC 地址。然而，在这个时间点上，我们还没有把包发送出去，所以先得搞清楚应该把包发给谁，这个只要查一下路由表就知道了。在路由表中找到相匹配的条目，然后把包发给 Gateway 列中的 IP 地址就可以了。

既然已经知道了包应该发给谁，那么只要将对方的 MAC 地址填上去就好了，但到这里为止根本没有出现对方的 MAC 地址，也就是说我们现在根本不知道对方的 MAC 地址是什么。因此，我们还需要执行根据 IP 地址查询 MAC 地址的操作。

通过 ARP 查询目标路由器的 MAC 地址

这里我们需要使用 ARP，它其实非常简单。在以太网中，有一种叫作广播的方法，可以把包发给连接在同一以太网中的所有设备。ARP 就是利用广播对所有设备提问：“×× 这个 IP 地址是谁的？请把你的 MAC 地址告诉我。”然后就会有人回答：“这个 IP 地址是我的，我的 MAC 地址是××××。”

<figure> </figure>

如果对方和自己处于同一个子网中，那么通过上面的操作就可以得到对方的 MAC 地址。然后，我们将这个 MAC 地址写入 MAC 头部，MAC 头部就完成了。

不过，如果每次发送包都要这样查询一次，网络中就会增加很多 ARP 包，因此我们会将查询结果放到一块叫作 ARP 缓存的内存空间中留着以后用。也就是说，在发送包时，先查询一下 ARP 缓存，如果其中已经保存了对方的 MAC 地址，就不需要发送 ARP 查询，直接使用 ARP 缓存中的地址，而当 ARP 缓存中不存在对方 MAC 地址时，则发送 ARP 查询。显示 ARP 缓存的方法和 MAC 地址的 cmd 指令为 `arp -a`

有了 ARP 缓存，我们可以减少 ARP 包的数量，但如果总是使用 ARP 缓存中保存的地址也会产生问题。例如当 IP 地址发生变化时，ARP 缓存的内容就会和现实发生差异。为了防止这种问题的发生，ARP 缓存中的值在经过一段时间后会删除，一般这个时间在几分钟左右。这个删除的操作非常简单粗暴，不管 ARP 缓存中的内容是否有效，只要经过几分钟就全部删掉，这样就不会出问题了。当地址从 ARP 缓存中删除后，只要重新执行一次 ARP 查询就可以再次获得地址了。

上面这个策略能够在几分钟后消除缓存和现实的差异，但 IP 地址刚刚发生改变的时候，ARP 缓存中依然会保留老的地址，这时就会发生通信的异常。

将 MAC 头部加在 IP 头部的前面，整个包就完成了。到这里为止，整个打包的工作是由 IP 模块负责的。有人认为，MAC 头部是以太网需要的内容，并不属于 IP 的职责范围，但从现实来看，让 IP 负责整个打包工作是有利的。如果在交给网卡之前，IP 模块能够完成整个打包工作，那么网卡只要将打好的包发送出去就可以了。对于除 IP 以外的其他类型的包也是一样，如果在交给网卡之前完成打包，那么对于网卡来说，发送的操作和发送 IP 包是完全相同的。这样一来，同一块网卡就可以支持各种类型的包。至于接收操作，我们到后面会讲，但如果接收的包可以原封不动直接交给 IP 模块来

处理，网卡就只要负责接收就可以了。这样一来，一块网卡也就能支持各种类型的包了。与其机械地设计模块和设备之间的分工，导致网卡只能支持 IP 包，不如将分工设计得现实一些，让网卡能够灵活支持各种类型的包。

以太网的基本知识

完成 IP 模块的工作之后，下面就该轮到网卡了，不过在此之前，我们先来了解一些以太网的基本知识。

以太网是一种为多台计算机能够彼此自由和廉价地相互通信而设计的通信技术，它的原型如下图(a)所示。

<figure> </figure>

从图上不难看出，这种网络的本质其实就是一根网线。图上还有一种叫作收发器的小设备，它的功能只是将不同网线之间的信号连接起来而已。因此，当一台计算机发送信号时，信号就会通过网线流过整个网络，最终到达所有的设备。这就好像所有人待在一个大房间里，任何一个人说话，所有人都能够听到，同样地，这种网络中任何一台设备发送的信号所有设备都能接收到。不过，我们无法判断一个信号到底是发给谁的，因此需要在信号的开头加上接收者的信息，也就是地址。这样一来就能够判断信号的接收者了，与接收者地址匹配的设备就接收这个包，其他的设备则丢弃这个包，这样我们的包就送到指定的目的地了。为了控制这一操作，我们就需要使用前面表里列出的 MAC 头部。通过 MAC 头部中的接收方 MAC 地址，就能够知道包是发给谁的；而通过发送方 MAC 地址，就能够知道包是谁发出的；此外，通过以太类型就可以判断包里面装了什么类型的内容。以太网其实就这么简单。

这个原型后来变成了图（b）中的结构。这个结构是将主干网线替换成了一个中继式集线器，将收发器网线替换成了双绞线。不过，虽然网络的结构有所变化，但信号会发送给所有设备这一基本性质并没有改变。

后来，图（c）这样的使用交换式集线器的结构普及开来，现在我们说的以太网指的都是这样的结构。这个结构看上去和（b）很像，但其实里面有一个重要的变化，即信号会发送给所有设备这一性质变了，现在信号只会流到根据 MAC 地址指定的设备，而不会到达其他设备了。当然，根据 MAC 地址来传输包这一点并没有变，因此 MAC 头部的设计也得以保留。

尽管以太网经历了数次变迁，但其基本的 3 个性质至今仍未改变，即将包发送到 MAC 头部的接收方 MAC 地址代表的目的地，用发送方 MAC 地址识别发送方，用以太类型识别包的内容。因此，大家可以认为具备这 3 个性质的网络就是以太网。

以太网中的各种设备也是基于以太网规格来工作的，因此下面的内容不仅适用于客户端计算机，同样也适用于服务器、路由器等各种设备。此外，以太网和 IP 一样，并不关心网络包的实际内容，因此以太网的收发操作也和 TCP 的工作阶段无关，都是共通的。

将 IP 包转换成电或光信号发送出去

下面来看看以太网的包收发操作。IP 生成的网络包只是存放在内存中的一串数字信息，没有办法直接发送给对方。因此，我们需要将数字信息转换为电或光信号，才能在网线上传输，也就是说，这才是真正的数据发送过程。

负责执行这一操作的是网卡，但网卡也无法单独工作，要控制网卡还需要网卡驱动程序。驱动程序不只有网卡才有，键盘、鼠标、显卡、声卡等各种硬件设备都有。当然，不同厂商和型号的网卡在结构上有所不同，因此网卡驱动程序也是厂商开发的专用程序。

网卡的内部结构如图所示，这是一张网卡主要构成要素的概念图，并不代表硬件的实际结构，但依然可以看清大体的思路。

<figure> </figure>

记住这一内部结构之后，我们再来介绍包收发的操作过程，现在，我们先来讲讲网卡的初始化过程。

网卡并不是通上电之后就可以马上开始工作的，而是和其他硬件一样，都需要进行初始化。也就是说，打开计算机启动操作系统的时候，网卡驱动程序会对硬件进行初始化操作，然后硬件才进入可以使用的状态。这些操作包括硬件错误检查、初始设置等步骤，这些步骤对于很多其他硬件也是共通的，但也有一些操作是以太网特有的，那就是在控制以太网收发操作的 MAC 模块中设置 MAC 地址。

网卡的 ROM 中保存着全世界唯一的 MAC 地址，这是在生产网卡时写入的，将这个值读出之后就可以对 MAC 模块进行设置，MAC 模块就知道自己对应的 MAC 地址了。也有一些特殊的方法，比如从命令或者配置文件中读取 MAC 地址并分配给 MAC 模块。这种情况下，网卡会忽略 ROM 中的 MAC 地址。有人认为在网卡通电之后，ROM 中的 MAC 地址就自动生效了，其实不然，真正生效的是网卡驱动进行初始化时在 MAC 模块中设置的那个 MAC 地址。在操作系统启动并完成这些初始化操作之后，网卡就可以等待来自 IP 的委托了。

给网络包再加 3 个控制数据

好了，下面来看一看网卡是如何将包转换成电信号并发送到网线中的。网卡驱动从 IP 模块获取包之后，会将其复制到网卡内的缓冲区中，然后向 MAC 模块发送发送包的命令。接下来就轮到 MAC 模块进行工作了。

首先，MAC 模块会将包从缓冲区中取出，并在开头加上报头和起始帧分界符，在末尾加上用于检测错误的帧校验序列。

<figure> </figure>

报头是一串像 10101010... 这样 1 和 0 交替出现的比特序列，长度为 56 比特，它的作用是确定包的读取时机。当这些 1010 的比特序列被转换成电信号后，会形成如下图这样的波形。

<figure> </figure>

接收方在收到信号时，遇到这样的波形就可以判断读取数据的时机。关于这一块内容，我们得先讲讲如何通过电信号来读取数据。

<figure> </figure>

用电信号来表达数字信息时，我们需要让 0 和 1 两种比特分别对应特定的电压和电流，例如上图（a）这样的电信号就可以表达数字信息。然而，实际的信号并不像所示的那样有分隔每个比特的辅助线，因此在测量电压和电流时必须先判断出每个比特的界限在哪里。要解决这个问题，最简单的方法就是在数据信号之外再发送一组用来区分比特间隔的时钟信号。如上图（b）所示，当时钟信号从下往上变化时读取电压和电流的值，然后和 0 或 1 进行对应就可以了。但是这种方法存在问题。当距离较远，网线较长时，两条线路的长度会发生差异，数据信号和时钟信号的传输会产生时间差，时钟就会发生偏移。

要解决这个问题，可以采用将数据信号和时钟信号叠加在一起的方法。这样的信号如上图（c）所示，发送方将这样的信号发给接收方。由于时钟信号是像（b）这样按固定频率进行变化的，只要能够找到这个变化的周期，就可以从接收到的信号（c）中提取出时钟信号（b），进而通过接收信号（c）和时钟信号（b）计算出数据信号（a），这和发送方将数据信号和时钟信号进行叠加的过程正好相反。然后，只要根据时钟信号（b）的变化周期，我们就可以从数据信号（a）中读取相应的电压和电流值，并将其还原为 0 或 1 的比特了。

这里的重点在于如何判断时钟信号的变化周期。时钟信号是以 10 Mbit/s 或者 100 Mbit/s 这种固定频率进行变化的，就像我们乘坐自动扶梯一样，只要对信号进行一段时间的观察，就可以找到其变化的周期。因此，我们不能一开始就发送包的数据，而是要在前面加上一段用来测量时钟信号的特殊信号，这就是报头的作用。

以太网根据速率和网线类型的不同分为多种派生方式，每种方式的信号形态也有差异，并不都是像本例中讲的这样，单纯通过电压和电流来表达 0 和 1 的。因此，101010...这样的报头数字信息在转换成电信号后，其波形也不一定是前面那个样子，而是根据方式的不同而不同。但是，报头的作用和基本思路是一致的。

报头后面的起始帧分界符在前面的图里也画出来了，它的末尾比特排列有少许变化。接收方以这一变化作为标记，从这里开始提取网络包数据。也就是说，起始帧分界符是一个用来表示包起始位置的标记。

末尾的 FCS（帧校验序列）用来检查包传输过程中因噪声导致的波形紊乱、数据错误，它是一串 32 比特的序列，是通过一个公式对包中从头到尾的所有内容进行计算而得出来的。具体的计算公式在此省略，它和磁盘等设备中使用的 CRC 错误校验码是同一种东西，当原始数据中某一个比特发生变化时，计算出来的结果就会发生变化。在包传输过程中，如果受到噪声的干扰而导致其中的数据发生了变化，那么接收方计算出的 FCS 和发送方计算出的 FCS 就会不同，这样我们就可以判断出数据有没有错误。

向集线器发送网络包

加上报头、起始帧分界符和 FCS 之后，我们就可以将包通过网线发送出去了。发送信号的操作分为两种，一种是使用集线器的半双工模式，另一种是使用交换机的全双工模式。

在半双工模式中，为了避免信号碰撞，首先要判断网线中是否存在其他设备发送的信号。如果有，则需要等待该信号传输完毕，因为如果在有信号时再发送一组信号，两组信号就会发生碰撞。当之前的信号传输完毕，或者本来就没有信号在传输的情况下，我们就可以开始发送信号了。首先，MAC 模块从报头开始将数字信息按每个比特转换成电信号，然后由 PHY，或者叫 MAU 的信号收发模块发送出去。在这里，将数字信息转换为电信号的速率就是网络的传输速率，例如每秒将 10 Mbit 的数字信息转换为电信号发送出去，则速率就是 10 Mbit/s。

接下来，PHY（MAU）模块会将信号转换为可在网线上传输的格式，并通过网线发送出去。以太网规格中对不同的网线类型和速率以及其对应的信号格式进行了规定，但 MAC 模块并不关心这些区别，而是将可转换为任意格式的通用信号发送给 PHY（MAU）模块，然后 PHY（MAU）模块再将其转换为可在网线上传输的格式。大家可以认为 PHY（MAU）模块的功能就是对 MAC 模块产生的信号进行格式转换。当然，以太网还有很多不同的派生方式，网线传输的信号格式也有各种变化。此外，实际在网线中传输的信号很复杂，我们无法一一介绍，但是如果一点都不讲，大家可能对此难以形成一个概念，所以就举一个例子，大家感受一下就好。图 2.27 就是这样一个例子，我们这里就不详细解释了，总之，网线中实际传输的信号就是这个样子的。

<figure> </figure>

PHY（MAU）的职责并不是仅仅是将 MAC 模块传递过来的信号通过网线发送出去，它还需要监控接收线路中有没有信号进来。在开始发送信号之前，需要先确认没有其他信号进来，这时才能开始发送。如果在信号开始发送到结束发送的这段时间内一直没有其他信号进来，发送操作就成功完成了。以太网不会确认发送的信号对方有没有收到。根据以太网的规格，两台设备之间的网线不能超过 100 米，在这个距离内极少会发生错误，万一发生错误，协议栈的 TCP 也会负责搞定，因此在发送信号时没有必要检查错误。

在发送信号的过程中，接收线路不应该有信号进来，但情况并不总是尽如人意，有很小的可能性出现多台设备同时进行发送操作的情况。如果有其他设备同时发送信号，这些信号就会通过接收线路传进来。

在使用集线器的半双工模式中，一旦发生这种情况，两组信号就会相互叠加，无法彼此区分出来，这就是所谓的信号碰撞。这种情况下，继续发送信号是没有意义的，因此发送操作会终止。为了通知其他设备当前线路已发生碰撞，还会发送一段时间的阻塞信号，然后所有的发送操作会全部停止。

等待一段时间之后，网络中的设备会尝试重新发送信号。但如果所有设备的等待时间都相同，那肯定还会发生碰撞，因此必须让等待的时间相互错开。具体来说，等待时间是根据 MAC 地址生成一个随机数计算出来的。

当网络拥塞时，发生碰撞的可能性就会提高，重试发送的时候可能又会和另外一台设备的发送操作冲突，这时会将等待时间延长一倍，然后再次重试。以此类推，每次发生碰撞就将等待时间延长一

倍，最多重试 10 次，如果还是不行就报告通信错误。

另一种全双工模式我们会在后面探索交换机时进行介绍，在全双工模式中，发送和接收可以同时进行，不会发生碰撞。因此，全双工模式中不需要像半双工模式这样考虑这么多复杂的问题，即便接收线路中有信号进来，也可以直接发送信号。

接收返回包

网卡将包转换为电信号并发送出去的过程到这里就结束了，既然讲到了以太网的工作方式，那我们不妨继续看看接收网络包时的操作过程。

在使用集线器的半双工模式以太网中，一台设备发送的信号会到达连接在集线器上的所有设备。这意味着无论是不是发给自己的信号都会通过接收线路传进来，因此接收操作的第一步就是不管三七二十一把这些信号全都收进来再说。

信号的开头是报头，通过报头的波形同步时钟，然后遇到起始帧分界符时开始将后面的信号转换成数字信息。这个操作和发送时是相反的，即PHY（MAU）模块先开始工作，然后再轮到 MAC 模块。首先，PHY（MAU）模块会将信号转换成通用格式并发送给 MAC 模块，MAC 模块再从头开始将信号转换为数字信息，并存放到缓冲区中。当到达信号的末尾时，还需要检查 FCS。具体来说，就是将从包开头到结尾的所有比特套用到公式中计算出 FCS，然后和包末尾的 FCS 进行对比，正常情况下两者应该是一致的，如果中途受到噪声干扰而导致波形发生紊乱，则两者的值会产生差异，这时这个包就会被当作错误包而被丢弃。

如果 FCS 校验没有问题，接下来就要看一下 MAC 头部中接收方 MAC 地址与网卡在初始化时分配给自己的 MAC 地址是否一致，以判断这个包是不是发给自己的。我们没必要去接收发给别人的包，因此如果不是自己的包就直接丢弃，如果接收方 MAC 地址和自己 MAC 地址一致，则将包放入缓冲区中。到这里，MAC 模块的工作就完成了，接下来网卡会通知计算机收到了一个包。

通知计算机的操作会使用一个叫作中断的机制。在网卡执行接收包的操作的过程中，计算机并不是一直监控着网卡的活动，而是去继续执行其他的任务。因此，如果网卡不通知计算机，计算机是不知道包已经收到了这件事的。网卡驱动也是在计算机中运行的一个程序，因此它也不知道包到达的状态。在这种情况下，我们需要一种机制能够打断计算机正在执行的任务，让计算机注意到网卡中发生的事情，这种机制就是中断。

具体来说，中断的工作过程是这样的。首先，网卡向扩展总线中的中断信号线发送信号，该信号线通过计算机中的中断控制器连接到 CPU。当产生中断信号时，CPU 会暂时挂起正在处理的任务，切换到操作系统中的中断处理程序。然后，中断处理程序会调用网卡驱动，控制网卡执行相应的接收操作。

中断是有编号的，网卡在安装的时候就在硬件中设置了中断号，在中断处理程序中则将硬件的中断号和相应的驱动程序绑定。例如，假设网卡的中断号为 11，则在中断处理程序中将中断号 11 和相应的网卡驱动绑定起来，当网卡发起中断时，就会自动调用网卡驱动了。现在的硬件设备都遵循即插

即用规范自动设置中断号，我们没必要去关心中断号了，在以前需要手动设置中断号的年代，经常发生因为设置了错误的中断号而导致网卡无法正常工作的问题。

网卡驱动被中断处理程序调用后，会从网卡的缓冲区中取出收到的包，并通过 MAC 头部中的以太类型字段判断协议的类型。现在我们在大多数情况下都是使用 TCP/IP 协议，但除了 TCP/IP 之外还有很多其他类型的协议，例如 NetWare 中使用的 IPX/SPX，以及 Mac 电脑中使用的 AppleTalk 等协议。这些协议都被分配了不同的以太类型，如 0080（十六进制）代表 IP 协议，网卡驱动就会把这样的包交给 TCP/IP 协议栈；如果是 809B 则表示 AppleTalk 协议，就把包交给 AppleTalk 协议栈，以此类推。

按照探索之旅的思路，大家可能会认为向 Web 服务器发送包之后，后面收到的一定是 Web 服务器返回的包，其实并非如此。计算机中同时运行了很多程序，也会同时进行很多通信操作，因此收到的包也有可能是其他应用程序的。不过，即便如此也没问题，网卡不会关心包里的内容，只要按照以太类型将包交给对应的协议栈就可以了。接下来，协议栈会判断这个包应该交给哪个应用程序，并进行相应的处理。

将服务器的响应包从 IP 传递给 TCP

下面我们假设 Web 服务器返回了一个网络包，那么协议栈会进行哪些处理呢？服务器返回的包的以太类型应该是 0800，因此网卡驱动会将其交给 TCP/IP 协议栈来进行处理。接下来就轮到 IP 模块开始工作了，第一步是检查 IP 头部，确认格式是否正确。如果格式没有问题，下一步就是查看接收方 IP 地址。如果接收网络包的设备是一台 Windows 客户端计算机，那么服务器返回的包的接收方 IP 地址应该与客户端网卡的地址一致，检查确认之后我们就可以接收这个包了。

如果接收方 IP 地址不是自己的地址，那一定是发生了什么错误。客户端计算机不负责对包进行转发，因此不应该收到不是发给自己的包。当发生这样的错误时，IP 模块会通过 ICMP 消息将错误告知发送方（第一个图）。ICMP 规定了各种类型的消息，如下表所示。

<figure> </figure>

当我们遇到这个错误时，IP 模块会通过 Destination unreachable 消息通知对方。从这张表的内容中我们可以看到在包的接收和转发过程中能够遇到的各种错误。

如果接收方 IP 地址正确，则这个包会被接收下来，这时还需要完成另一项工作。IP 协议有一个叫作分片的功能，具体的内容我们将在第 3 章探索路由器时进行介绍。简单来说，网线和局域网中只能传输小包，因此需要将大的包切分成多个小包。如果接收到的包是经过分片的，那么 IP 模块会将它们还原成原始的包。分片的包会在 IP 头部的标志字段中进行标记，当收到分片的包时，IP 模块会将其暂存在内部的内存空间中，然后等待 IP 头部中具有相同 ID 的包全部到达，这是因为同一个包的所有分片都具有相同的 ID。此外，IP 头部还有一个分片偏移量（fragment offset）字段，它表示当前分片在整个包中所处的位置。根据这些信息，在所有分片全部收到之后，就可以将它们还原成原始的包，这个操作叫作分片重组。

到这里，IP 模块的工作就结束了，接下来包会被交给 TCP 模块。TCP 模块会根据 IP 头部中的接收方和发送方 IP 地址，以及 TCP 头部中的接收方和发送方端口号来查找对应的套接字。找到对应的套接字之后，就可以根据套接字中记录的通信状态，执行相应的操作了。例如，如果包的内容是应用程序数据，则返回确认接收的包，并将数据放入缓冲区，等待应用程序来读取；如果是建立或断开连接的控制包，则返回相应的响应控制包，并告知应用程序建立和断开连接的操作状态。

UDP 协议收发

不需要重发的数据用 UDP 发送更高效

跟着第 1 章的脚步，本章我们探索了通过套接字收发数据的整个过程，这个过程到这里已经告一段落了。接下来，网络包会从计算机出来跑向集线器，这个过程我们将在下一章来介绍，现在先来说点题外话。

大多数的应用程序都像之前介绍的一样使用 TCP 协议来收发数据，但当然也有例外。有些应用程序不使用 TCP 协议，而是使用 UDP 协议来收发数据。向 DNS 服务器查询 IP 地址的时候我们用的也是 UDP 协议。下面就简单介绍一下 UDP 协议。

其实 TCP 中就包含了 UDP 的一些要点。TCP 的工作方式十分复杂，如果我们能够理解 TCP 为什么要设计得如此复杂，也就能够理解 UDP 了。那么，为什么要设计得如此复杂呢？因为我们需要将数据高效且可靠地发送给对方。为了实现可靠性，我们就需要确认对方是否收到了我们发送的数据，如果没有还需要再发一遍。

要实现上面的要求，最简单的方法是数据全部发送完毕之后让接收方返回一个接收确认。这样一来，如果没收到直接全部重新发送一遍就好了，根本不用像 TCP 一样要管理发送和确认的进度。但是，如果漏掉了一个包就要全部重发一遍，怎么看都很低效。为了实现高效的传输，我们要避免重发已经送达的包，而是只重发那些出错的或者未送达的包。TCP 之所以复杂，就是因为要实现这一点。

不过，在某种情况下，即便没有 TCP 这样复杂的机制，我们也能够高效地重发数据，这种情况就是数据很短，用一个包就能装得下。如果只有一个包，就不用考虑哪个包未送达了，因为全部重发也只不过是重发一个包而已，这种情况下我们就不需要 TCP 这样复杂的机制了。而且，如果不使用 TCP，也不需要发送那些用来建立和断开连接的控制包了。此外，我们发送了数据，对方一般都会给出回复，只要将回复的数据当作接收确认就行了，也不需要专门的接收确认包了。

控制用的短数据

这种情况就适合使用 UDP。像 DNS 查询等交换控制信息的操作基本上都可以在一个包的大小范围内解决，这种场景中就可以用 UDP 来代替 TCP。UDP 没有 TCP 的接收确认、窗口等机制，因此在收发数据之前也不需要交换控制信息，也就是说不需要建立和断开连接的步骤，只要在从应用程序获取的数据前面加上 UDP 头部，然后交给 IP 进行发送就可以了。

<figure> </figure>

接收也很简单，只要根据 IP 头部中的接收方和发送方 IP 地址，以及 UDP 头部中的接收方和发送方端口号，找到相应的套接字并将数据交给相应的应用程序就可以了。除此之外，UDP 协议没有其他功能了，遇到错误或者丢包也一概不管。因为 UDP 只负责单纯地发送包而已，并不像 TCP 一样会对包的送达状态进行监控，所以协议栈也不知道有没有发生错误。但这样并不会引发什么问题，因此出错时就收不到来自对方的回复，应用程序会注意到这个问题，并重新发送一遍数据。这样的操作本身并不复杂，也并不会增加应用程序的负担。

音频和视频数据

还有另一个场景会使用 UDP，就是发送音频和视频数据的时候。音频和视频数据必须在规定的时间内送达，一旦送达晚了，就会错过播放时机，导致声音和图像卡顿。如果像 TCP 一样通过接收确认响应来检查错误并重发，重发的过程需要消耗一定的时间，因此重发的数据很可能已经错过了播放的时机。一旦错过播放时机，重发数据也是没有用的，因为声音和图像已经卡顿了，这是无法挽回的。当然，我们可以用高速线路让重发的数据能够在规定的时间内送达，但这样一来可能要增加几倍的带宽才行。

此外，音频和视频数据中缺少了某些包并不会产生严重的问题，只是会产生一些失真或者卡顿而已，一般都是可以接受的。

在这些无需重发数据，或者是重发了也没什么意义的情况下，使用 UDP 发送数据的效率会更高。

本章我们探索了在收发数据时，操作系统中的协议栈是如何工作的，以及网卡是如何将包转换成电信号通过网线发送出去的。到这里，我们的网络包已经沿着网线流出了客户端计算机，下一章，我们将探索网络包如何经过集线器、交换机、路由器等设备，最终到达互联网。